

Arreglos

No.	Respuesta	Explicación:
1	b. <code>int[] arr = {1, 2, 3};</code> c. <code>int[] arr = new int[] {1, 2, 3};</code> d. <code>int[] arr = new int[3];</code>	Con <code>int[] arr = {1, 2, 3};</code> usamos una inicialización directa con los valores. Con <code>new int[] {1, 2, 3}</code> también creamos el arreglo indicando sus valores. Y con <code>new int[3]</code> creamos un arreglo con espacio para 3 elementos, que al ser <code>int</code> empiezan automáticamente en 0.
2	d. 15	Primero toma <code>array[0][0]</code> , que es 1; después <code>array[1][1]</code> , que es 5; y finalmente <code>array[2][2]</code> , que es 9. Entonces hacemos $1 + 5 + 9 = 15$.
3	c. 159	El <code>if</code> pregunta si <code>i == j</code> . Esto significa que solamente se imprimen los elementos donde el número de fila y columna son iguales. Entonces tenemos <code>[0][0] = 1</code> , <code>[1][1] = 5</code> y <code>[2][2] = 9</code> . Por eso se imprime 1 5 9.
4	c. 4 an <code>ArrayIndexOutOfBoundsException</code>	Primero <code>array[4]</code> es <code>{0,4,8,12,16}</code> , entonces la posición [1] contiene 4 y eso se imprime correctamente. Después intentamos acceder a <code>array[1][4]</code> , pero <code>array[1]</code> solamente tiene <code>{0,1}</code> , así que sus posiciones válidas son 0 y 1. Como intentamos acceder a la posición 4, Java lanza un <code>ArrayIndexOutOfBoundsException</code> .
5	b. 3true1	<code>array2D[0]</code> es <code>{0,1,2}</code> , así que su <code>length</code> es 3. Después <code>array2D[1]</code> es <code>{3,4,5,6}</code> , que sigue siendo un arreglo de tipo <code>int[]</code> , por eso <code>getClass().isArray()</code> devuelve <code>true</code> . Por último <code>array2D[0][1]</code> corresponde al segundo elemento de la primera fila, que es 1. Al juntarlo todo obtenemos 3true1.
6	a. 1	<code>Arrays.sort(fruts)</code> ordena las cadenas alfabéticamente, quedando <code>[Apple, Banana, Orange]</code> . Después <code>binarySearch()</code> busca "Banana" y encuentra que está en el índice 1. Por eso el resultado es 1.
7	b. [2, 1, 3, 4]	<code>Arrays.sort(strings, 1, 4)</code> solamente ordena las posiciones 1, 2 y 3. Originalmente esas posiciones tiene 3, 1, 4, y después de ordenarlas quedan como 1, 3, 4. El elemento de la posición 0, que es 2, no se modifica. Por eso obtenemos [2, 1, 3, 4].
8	e. 35	El arreglo comienza como <code>{1,2,3,4,5}</code> . <code>System.arraycopy(array, 2, array, 1, 2)</code> significa que copiamos 2 elementos empezando desde la posición 2 y los ponemos desde la posición 1. Los elementos que copiamos son 3 y 4, así que el arreglo termina como <code>{1,3,4,4,5}</code> . Por eso <code>array[1]</code> es 3 y <code>array[4]</code> sigue siendo 5, dando como salida 35.

9

a.int[][] array2D = new int[2][2]; array2D[0][0] = 1; array2D[0][1] = 2; array2D[1][0] = 3; array2D[1][1] = 4;

b.int[] array = {0, 1}; int[][][] array3D = new int[2][2][2]; array3D[0][0] = array; array3D[0][1] = array; array3D[1][0] = array; array3D[0][1] = array;

La opción a es válida porque new int[2][2] crea una matriz de 2 filas por 2 columnas. La opción b también es válida porque se crea un arreglo de tres dimensiones int[2][2][2] y cada array3D[i][j] puede almacenar un int[]. Las opciones c, d y e tienen problemas con las dimensiones o con la sintaxis.